Everything You Need to Know About REST: Semantics & Trade-offs

Decomposition of Fielding's Constraints, Richardson Maturity, HTTP Caching, and REST vs gRPC

EXECUTIVE SUMMARY

Representational State Transfer (REST) is an architectural style designed to decouple network clients and servers through uniform interfaces, stateless interactions, and standard HTTP semantics. While ubiquitous for external edge communication and web clients, internal microservice fabrics often suffer from JSON serialization overhead and lack of strict typing compared to binary RPCs.

1. ROY FIELDING'S 6 CORE ARCHITECTURAL CONSTRAINTS

1. Client-Server

Separates user concerns from backend data persistence, enabling independent scaling.

2. Statelessness

No client session stored on servers. Every request carries auth and context.

3. Cacheability

Responses declare cacheability via headers, offloading traffic to edge CDNs.

4. Uniform Interface

URIs identify resources; standard verbs manipulate state self-descriptively.

5. Layered System

Intermediary proxies, CDNs, and load balancers operate transparently.

6. Code on Demand

(Optional) Server extends client capability by sending executable JS / WASM.

2. HTTP VERBS: SAFE VS. IDEMPOTENT INVARIANTS

Method	Domain Semantics	Safe? (Read-Only)	Idempotent?	Status Code
GET	Fetch resource representation	YES	YES	200 OK
POST	Create subordinate entity	NO	NO	201 Created
PUT	Full resource replacement	NO	YES	200 OK / 204 No Content
PATCH	Partial delta update	NO	Conditional	200 OK
DELETE	Purge resource	NO	YES	200 OK / 204 No Content

3. THE RICHARDSON MATURITY MODEL



4. HTTP CACHING HEADERS: ETAG & CACHE-CONTROL

Conditional validation offloads read traffic to CDNs and browser caches without serving stale data:

```
// Conditional HTTP GET Interaction:
1. Client -> GET /api/v1/products/42
2. Server -> HTTP 200 OK
   ETag: "w/33a2f4ee24"
   Cache-Control: public, max-age=300, must-revalidate

3. After 300s, Client sends Conditional Request:
   GET /api/v1/products/42
   If-None-Match: "w/33a2f4ee24"

4. Server validates DB checksum -> HTTP 304 Not Modified (0-byte payload transferred!)
```

5. STANDARDIZED ERROR HANDLING: RFC 7807 PROBLEM DETAILS

```
{
  "type": "https://api.example.com/errors/insufficient-funds",
  "title": "Insufficient Account Balance",
  "status": 422,
  "detail": "Your balance of $12.50 is less than the transaction total of $30.00.",
  "instance": "/transactions/tx_88492"
}
```

6. NORTH-SOUTH VS. EAST-WEST: REST VS. GRPC

Architectures segment communication protocols based on topological network boundaries:

Traffic Boundary	Protocol of Choice	Architectural Rationale
North - South (Edge) Client → API Gateway	REST / JSON over HTTP	Native browser/mobile support; CDN caching; zero custom client proxy required.
East - West (Internal) Service ↔ Service	gRPC / Protobuf over HTTP/2	70% smaller payloads; 5x faster CPU parsing; multiplexed TCP; strict IDL contracts.

7. THE MICROSERVICES REST ANTI-PATTERN: CHATTY CASCADES

Under-Fetching Bottleneck

To render a single user profile screen, a mobile app must execute 6 separate REST roundtrips: `GET /user`, `GET /orders`, `GET /rewards`, etc. Solved by **BFF Pattern** or **GraphQL**.

JSON CPU Parsing Tax

At 50,000 req/sec across an internal microservice mesh, parsing ASCII JSON strings into heap objects consumes massive CPU cycles and triggers frequent garbage collection pauses.

8. PRODUCTION REST VERIFICATION CHECKLIST

REST API Production Guidelines

1. **Plural Nouns for URIs:** Use `/orders` and `/orders/{id}`; never verbs (e.g. avoid `/getOrderById`).
2. **Pagination Mandatory:** Every collection endpoint must enforce cursor or limit/offset pagination (`?limit=50&cursor=xyz`).
3. **Idempotency Header on POST:** Implement `Idempotency-Key` caching in Redis on payment and order creation endpoints.
4. **OpenAPI / Swagger Generation:** Maintain automated CI verification that code and OpenAPI specs do not drift.